

# Integrating Agentic AI into High-Performance Computing Workflows: A Two-Level Architecture for User-Governed and Institution-Governed Research Computing

## Abstract

High-performance computing (HPC) systems provide essential computational resources for modern scientific research but continue to present substantial usability barriers. Researchers must understand workload schedulers, resource requests, software environments, accelerators, storage systems, containers, and site-specific policies before effectively using an HPC system. Large language models (LLMs) and agentic artificial intelligence provide an opportunity to reduce these barriers by translating researcher intent into HPC operations. Directly connecting autonomous AI agents to production HPC systems, however, introduces significant concerns involving authorization, command execution, resource consumption, reproducibility, security, and accountability.

This paper presents a two-level adoption architecture for integrating agentic AI into research computing while maintaining existing HPC schedulers, authentication systems, software management tools, and security controls as authoritative components. The first level is a user-governed approach in which researchers provide persistent HPC-specific guidance to general-purpose coding agents through project-level Markdown instruction files. This approach can be deployed immediately in a researcher’s home or project directory and is intended to shape behavior rather than enforce authorization. The second level is an institution-governed approach in which a site-operated Model Context Protocol (MCP) gateway exposes narrowly defined, authenticated HPC capabilities such as documentation retrieval, software discovery, scheduler accounting, GPU utilization, Linux Pressure Stall Information (PSI), storage reporting, job validation, and controlled job submission. State-changing operations remain subject to external authorization and human approval.

We describe both adoption levels, representative researcher workflows, security boundaries, an implementation strategy using Slurm, Spack, containers, and site documentation, and an evaluation methodology that compares baseline AI assistance, Markdown-governed AI assistance, and MCP-integrated AI assistance. Recent ACM, IEEE, and PEARC work has already demonstrated LLM-based HPC support, MCP-mediated scientific interfaces, agentic scheduler execution, and AI-assisted resource prediction; accordingly, the contribution of this work is not MCP or LLM–HPC connectivity alone. Instead, we focus on a progressive governance model that distinguishes behavioral guidance from enforceable institutional controls and evaluates the incremental value of each layer. The proposed approach provides a path from immediately deployable user-level AI assistance to institutionally controlled agentic HPC while preserving existing administrative and scheduling controls.

**Keywords:** high-performance computing, HPC, large language models, artificial intelligence, AI agents, Model Context Protocol, MCP, Slurm, research computing, scientific workflows, resource optimization

# 1 Introduction

High-performance computing has become an important component of research across disciplines including engineering, computational science, artificial intelligence, data science, physics, chemistry, and the life sciences. While HPC systems provide substantial computational capability, effective use of those systems frequently requires researchers to understand concepts outside their primary scientific discipline. These include batch scheduling, CPU and memory allocation, accelerators, parallel computing, software environments, containers, storage architectures, and command-line Linux environments.

Research computing organizations address this problem through documentation, training, consultations, and direct researcher support. Nevertheless, researchers routinely encounter questions such as how much memory a job requires, which GPU should be requested, which software environment contains a particular package, why a Slurm job failed, or why a program performs poorly. Solving these problems frequently requires combining information from several sources, including source code, job scripts, scheduler accounting records, application logs, site documentation, software repositories, and performance monitoring systems.

Recent advances in large language models create an opportunity to provide a natural-language interface across these otherwise separate information sources. LLM-based agents extend this capability by allowing a model to invoke external tools rather than relying exclusively on information contained in its training data. The research landscape is developing rapidly: LLM-based systems have been evaluated for HPC user support [13, 15], scientific APIs have been exposed through MCP [7], HPC-native agents have been connected to Slurm execution [12], multi-agent systems have been proposed for HPC administration and software deployment [14, 8], and AI-based systems have been used for resource prediction and scheduling [6, 17]. These results establish the feasibility of AI-assisted HPC interaction but leave open the institutional question of how authority should be introduced progressively and measured.

Simply providing an AI agent with shell access to an HPC login node, however, is not an appropriate production architecture. An AI system capable of executing arbitrary commands could inadvertently modify data, consume excessive resources, disclose information, submit inappropriate workloads, or act beyond the intent of the researcher. Similar concerns arise when an agent is provided unrestricted access to scheduler interfaces.

This work proposes an alternative architecture in which AI does not replace the workload scheduler, research computing policies, or existing access controls. Instead, AI operates as a facilitation and orchestration layer above those systems.

The primary contributions of this paper are:

1. A two-level adoption model that separates an immediately deployable user-governed Markdown approach from an institution-governed MCP approach.
2. A portable project-level policy-file mechanism that communicates site behavioral expectations to multiple AI coding clients without requiring a new cluster service.
3. An HPC-specific MCP architecture that exposes narrow, policy-controlled capabilities instead of arbitrary shell access.
4. Integration of scheduler accounting, GPU telemetry, and Linux Pressure Stall Information (PSI) into AI-assisted resource recommendations.
5. A human-governed mechanism for transitioning from AI-generated recommendations to actions such as job submission.

6. An evaluation framework that compares baseline, user-level, and system-level AI integration for correctness, usability, resource efficiency, behavioral compliance, and externally enforced security.

The objective is not to create an autonomous replacement for the HPC scheduler or research computing facilitator. Rather, the objective is to augment existing research computing services with an interface capable of understanding researcher intent while remaining subject to established HPC controls.

## 2 Background and Related Work

The literature relevant to AI-assisted research computing now spans user support, agentic scientific interfaces, scheduler interaction, software automation, and resource optimization. This section positions the proposed two-level governance architecture relative to recent ACM, IEEE, and PEARC work.

### 2.1 LLM-Based HPC User Support

LLMs have increasingly been investigated as interfaces for HPC users. HPC-LLM combines retrieval and domain adaptation for operational questions involving Slurm, MPI, GPUs, filesystems, and troubleshooting [5]. A PEARC 2026 comparative study explicitly evaluates LLM-enabled HPC user support, reflecting the shift from proof-of-concept chatbots toward comparative operational evaluation [13]. Pasupuleti et al. similarly describe an IEEE-published LLM chatbot intended to automate common HPC support interactions and evaluate accuracy and usability [15].

These systems motivate our user-facing goals, but our evaluation asks a different question: how much additional value is obtained when a general-purpose assistant is first given persistent site behavioral instructions and then given authenticated, enforceable institutional context.

### 2.2 Agentic Interfaces, MCP, and HPC Execution

MCP has already emerged as an interface between AI agents and scientific or HPC services. OpenBridge exposes scientific APIs through an MCP-compliant server, allowing assistants to invoke domain tools through natural language [7]. BioCodex goes further in an HPC-native genomics setting, using MCP-oriented tooling and asynchronous Slurm execution for agentic workflows [12]. CoMAS-HPC proposes a collaborative multi-agent system for HPC administration and uses MCP to standardize access to HPC resources [14]. An IEEE contribution has also demonstrated an MCP server for execution in hybrid quantum-HPC environments [18].

These works mean that neither “MCP for science” nor “MCP connected to HPC execution” should be treated as novel by itself. The contribution proposed here is the explicit separation between a low-barrier user governance layer and an institutionally enforced MCP layer, together with an experiment designed to quantify the incremental benefit and failure containment provided by each.

### 2.3 Workflow, Software, and Scheduler Automation

Agentic HPC research also addresses software engineering and workload orchestration. Mondesire et al. use an LLM-based multi-agent system to automate HPC software compilation, deployment, and error resolution [8]. LangChain-Parsl connects LLM agent tool execution to HPC resources

so computational work can be delegated while the model orchestrates the higher-level workflow [1]. VibeCodeHPC explores coding-agent-driven performance optimization on supercomputers and employs Markdown-format configuration documents to define agent roles and operational rules [4]. Hoshino et al. provide a separate Claude Code GPU acceleration case study in which an instruction Markdown file is used to direct parts of the automated workflow [19].

Natural-language scheduling is also appearing outside traditional batch-HPC interfaces. Sliwko and Mizeria-Pietraszko demonstrate semantic workload allocation using an LLM to interpret natural-language placement preferences for a cluster scheduler [17]. Kwon et al. propose an ontology-driven LLM service protocol for HPC platforms with an emphasis on workflow interoperability and provenance-aware automation [16].

These systems demonstrate that agents can receive operational instructions and can mediate scheduling or workflow actions. Our user-level approach differs by treating the Markdown instruction mechanism as a deliberately limited *behavioral governance layer* that can be deployed before any privileged integration is available, rather than as an autonomous execution framework.

## 2.4 Resource Recommendation, GPU Telemetry, and Observability

AI-assisted resource selection predates the recent wave of agentic interfaces. The HARP framework uses execution history to estimate HPC application resource requirements and recommend more efficient allocations [6]. Related work has continued to examine AI-driven scheduling and runtime/resource prediction. This establishes that resource recommendation itself is not unique to an LLM agent.

GPU telemetry provides an important empirical basis for such recommendations. Sencan et al. analyze GPU utilization and GPU-memory behavior in large-scale HPC workloads using job metadata and accelerator telemetry, showing substantial opportunities to reason about utilization and provisioning at job level [9]. The ACCESS Software Documentation Service likewise demonstrates the value of structured, site-specific software information for helping HPC users identify available software resources [10], while Open OnDemand work illustrates institutional approaches for presenting AI workflows and training environments to researchers [11].

The present work extends this operational evidence layer by combining scheduler accounting and GPU telemetry with Linux Pressure Stall Information (PSI) for CPU, memory, and I/O. PSI is an established Linux systems measure and is not claimed as a new metric. The research question is whether exposing stall pressure alongside scheduler and accelerator data improves an AI assistant’s diagnosis and resource recommendations.

## 2.5 User-Level Markdown Instructions

General-purpose coding agents increasingly support persistent project instructions. Claude Code, Codex, Gemini CLI, and GitHub Copilot provide project or repository mechanisms that can carry site-specific guidance [27, 28, 29, 30]. Recent HPC agent studies also provide precedent for Markdown-formatted operational or role configuration [4, 19].

However, configuration text remains model context rather than an authorization mechanism. This motivates the distinction central to the proposed architecture: Markdown instructions can reduce inappropriate plans and recommendations, while MCP, Slurm, Linux permissions, and human approval gates independently enforce what actions are allowed.

## 2.6 Positioning and Research Gap

Table 1 summarizes the closest work and the dimensions relevant to this paper. The table is intentionally focused on architectural features rather than claiming that every cited system was designed for institutional governance.

Table 1: Feature-level comparison with selected related work. “–” indicates that the feature is not a primary component of the cited work; “partial” denotes a related but differently scoped mechanism.

| Work                         | User port  | sup- MD<br>port ance | guid- MCP  | Scheduler/jobs          | Performance data             | External enforcement              |
|------------------------------|------------|----------------------|------------|-------------------------|------------------------------|-----------------------------------|
| HARP [6]                     | –          | –                    | –          | Resource rec.           | Execution history            | –                                 |
| OpenBridge [7]               | Indirect   | –                    | Yes        | –                       | –                            | Tool boundary                     |
| BioCodex [12]                | Domain     | –                    | Yes        | Slurm                   | Job/workflow                 | Workflow controls                 |
| CoMAS-HPC [14]               | Admin.     | –                    | Yes        | Yes                     | System context               | Agent/tool controls               |
| LLM HPC support [13]         | Yes        | –                    | –          | Support context         | Evaluation data              | –                                 |
| Agent coding studies [4, 19] | Indirect   | Yes                  | Partial    | Batch execution         | Build/GPU tests              | Agent rules                       |
| GPU utilization [9]          | –          | –                    | –          | Job metadata            | GPU + memory                 | –                                 |
| <b>Proposed work</b>         | <b>Yes</b> | <b>Yes</b>           | <b>Yes</b> | <b>Slurm + approval</b> | <b>CPU, memory, GPU, PSI</b> | <b>MCP + HPC controls + human</b> |

The gap addressed in this paper is therefore not whether an LLM can answer HPC questions, invoke MCP tools, submit work to a scheduler, or recommend resources. Rather, we study a progressive institutional adoption model with three experimentally separable conditions: (A) general-purpose AI, (B) the same class of AI with portable HPC Markdown behavioral instructions, and (C) Markdown guidance plus an authenticated MCP service that supplies authoritative site context, performance telemetry, and enforceable operation boundaries. This framing permits behavioral compliance and technical enforcement to be measured separately.

## 2.7 Agent-to-Agent Communication

The Agent2Agent protocol addresses a related but different problem. Whereas MCP primarily connects an agent to resources and tools, A2A provides a standard for communication and collaboration between agents [21]. A2A therefore remains a potential later-stage component of the proposed architecture. Specialized software, scheduler, performance, and data agents could communicate with one another while each uses controlled MCP interfaces to interact with underlying services. The initial implementation and evaluation proposed in this paper do not require a multi-agent architecture.

## 3 Two-Level Adoption Model

The proposed architecture distinguishes between two practical levels of AI adoption. The levels are complementary rather than mutually exclusive. A research computing organization can begin with the user-level approach without deploying new infrastructure and later add the system-level approach for authenticated context, telemetry, and controlled actions.

### 3.1 Level 1: User-Governed AI with Markdown Policy Files

The user-level approach places an AI coding assistant in the researcher’s existing working environment and provides persistent HPC-specific instructions through Markdown files recognized by the selected client. A site may maintain one canonical policy source, such as `HPC-AI-INSTRUCTIONS.md`, and copy or render that content into `CLAUDE.md`, `AGENTS.md`, `GEMINI.md`, or `.github/copilot-instructions.md` as appropriate [27, 28, 29, 30].

This level requires no new scheduler integration. The agent can explain commands, generate Slurm scripts, review source code, suggest software environments, and help interpret errors while the researcher remains responsible for reviewing commands and initiating consequential actions. The Markdown file establishes expected behavior such as using Slurm for sustained compute, avoiding destructive filesystem operations, protecting credentials, and seeking confirmation before job submission or cancellation.

The primary advantage is deployability: the model can be made more HPC-aware inside a normal user project with little or no cluster-side engineering. The limitation is equally important: the policy file is a soft control. It influences model behavior but does not create a security boundary.

### 3.2 Level 2: Institution-Governed AI with MCP

The system-level approach introduces an institutional MCP gateway between the AI client and production HPC services. Instead of allowing the agent to construct arbitrary shell commands, the gateway exposes constrained operations such as `job_status()`, `software_find()`, `gpu_utilization()`, `psi_io()`, `validate_job()`, and, at higher trust levels, `submit_job()`.

This level adds authenticated site context, centralized policy, auditability, and enforceable authorization. The MCP service can check researcher identity, ownership, allocation membership, parameter ranges, approval state, and other policy conditions before an action reaches Slurm or another production service. The Markdown policy layer can remain in place because it reduces undesirable behavior before a tool call is attempted, while the MCP gateway independently prevents prohibited actions.

The two-level model provides a migration path rather than requiring an institution to begin with fully autonomous agents. User-level guidance can be introduced first, measured, and refined. MCP capabilities can then be added selectively where institutional context or enforceable action is valuable.

Table 2: Comparison of the two proposed AI adoption levels.

| Characteristic   | User-Level                                | Markdown         | Ap-<br>proach       | System-Level MCP Approach                             |
|------------------|-------------------------------------------|------------------|---------------------|-------------------------------------------------------|
| Deployment       | Researcher                                | home/project     | direc-<br>tory      | Institution-operated service                          |
| Primary purpose  | Behavioral                                | guidance         | and assis-<br>tance | Authenticated context and en-<br>forced capabilities  |
| Cluster changes  | None or minimal                           |                  |                     | MCP gateway and service integra-<br>tions             |
| Typical access   | Local files and normal user com-<br>mands |                  |                     | Narrow site-defined tools and re-<br>sources          |
| Policy strength  | Soft /                                    | behavior shaping |                     | Enforceable outside the model                         |
| Telemetry        | User-provided or locally visible          |                  |                     | Scheduler, GPU, PSI, storage, and<br>site telemetry   |
| Actions          | Researcher reviews and executes           |                  |                     | Controlled actions with validation<br>and approval    |
| Auditability     | Primarily local/client dependent          |                  |                     | Centralized institutional logging<br>possible         |
| Best initial use | Immediate researcher assistance           |                  |                     | Production integration and con-<br>trolled automation |

## 4 Design Principles

The proposed architecture is based on seven principles.

### 4.1 Existing HPC Infrastructure Remains Authoritative

The AI system does not become a scheduler. Slurm continues to determine resource availability, priorities, limits, accounting, and job execution.

Similarly, existing Linux permissions, authentication systems, allocation policies, storage permissions, and software controls remain authoritative.

AI proposes and orchestrates operations within these systems rather than bypassing them.

### 4.2 Least Capability

AI agents should receive the minimum capability necessary to perform a requested operation.

For example, an AI system that needs to determine whether a researcher’s job is running requires access to that researcher’s scheduler status, not unrestricted shell access.

The preferred abstraction is therefore:

Table 3: Proposed AI trust levels for HPC integration.

| Level | Capability     | Example                                         |
|-------|----------------|-------------------------------------------------|
| 1     | Advisory       | Generate a Slurm script                         |
| 2     | Read-only      | Retrieve job history or performance information |
| 3     | State-changing | Submit or cancel a job                          |

```
job_status(user)
```

rather than:

```
shell(command)
```

This distinction dramatically reduces the operational authority delegated to the model.

### 4.3 Progressive Trust

AI capabilities are divided into three trust levels.

Level 1 functionality can generally be introduced with minimal infrastructure changes.

Level 2 requires authenticated access to institutional information.

Level 3 requires substantially stronger authorization, logging, validation, and confirmation controls.

### 4.4 Human-Governed Actions

The transition between recommendation and execution is an explicit architectural boundary.

An AI assistant may generate the following recommendation:

```
1 GPU
8 CPU cores
64 GB RAM
2-hour time limit
```

and produce the corresponding Slurm script.

Submission should initially require an explicit researcher action such as:

```
Submit this job.
```

This model retains the convenience of natural-language interaction while making consequential actions visible to the researcher.

### 4.5 Progressive Adoption

An institution should not need to deploy a production AI gateway before researchers can benefit from AI assistance. The architecture therefore treats user-level Markdown guidance as a legitimate first deployment stage rather than merely a configuration detail of the final system. Capabilities can progress from user-governed advisory assistance to read-only institutional context, performance telemetry, and finally controlled state-changing operations. Each stage can be evaluated before additional authority is introduced.

## 4.6 Defense in Depth Across Client and Server

The architecture separates *behavioral guidance* from *enforcement*. Project-level configuration files instruct the AI client how it should behave in the HPC environment. The MCP gateway and underlying HPC services independently determine what the client is actually allowed to do.

This distinction is important because a model may misunderstand, omit, or disregard a project instruction. A researcher may also modify a local configuration file. Therefore, a rule such as “do not submit jobs without confirmation” should appear in the client instructions to reduce accidental behavior, but the submission tool should also require an authenticated request, validate parameters, and enforce the approval state. The result is a defense-in-depth model in which client configuration reduces unsafe intent before it reaches the service boundary and server-side controls prevent prohibited actions even when client guidance fails.

## 4.7 Evidence-Based Recommendations

The AI should preferentially use operational evidence rather than generic HPC recommendations.

Resource recommendations can incorporate:

- previous job runtime;
- requested versus consumed memory;
- CPU utilization;
- GPU compute utilization;
- GPU memory utilization and peak memory consumption;
- GPU power and accelerator saturation indicators where available;
- scheduler state;
- application exit status;
- filesystem behavior;
- Linux Pressure Stall Information (PSI) for CPU, memory, and I/O;
- PSI short-, medium-, and long-window averages (`avg10`, `avg60`, and `avg300`);
- cumulative stall time;
- application output.

Consequently, the AI assistant can evolve from a generic question-answering system into an HPC-specific optimization service.

## 5 Proposed Architecture

Figure 1 shows the two adoption paths. The user-level path can operate independently using existing user permissions and HPC commands. The system-level path inserts an MCP gateway that becomes the enforceable interface to institutional services. Both paths may use the same canonical Markdown policy content.

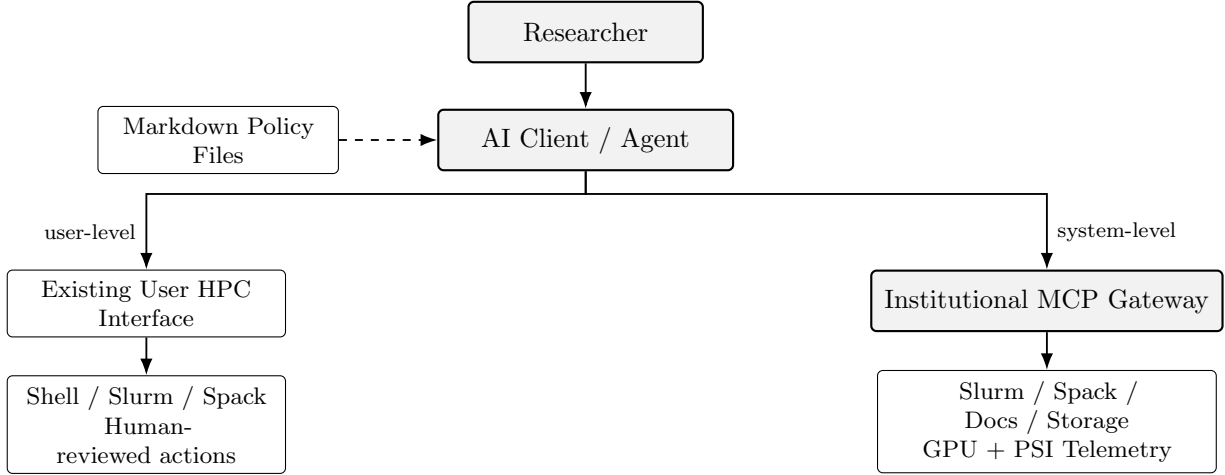


Figure 1: Two-level AI adoption architecture for HPC. The user-level path combines project Markdown instructions with existing researcher permissions and human-reviewed actions. The system-level path uses an institutional MCP gateway to provide authenticated context, telemetry, and enforceable controls.

At the user level, the AI client behaves like an HPC-aware assistant operating under the same permissions and responsibilities as the researcher. The Markdown policy file is intended to make appropriate site behavior explicit before commands are proposed. At the system level, the AI client communicates through site-defined MCP operations whose parameters, permissions, and approval requirements are enforced outside the model.

This separation allows the same research workflow to evolve without changing the user’s conceptual interface. A researcher might first ask an AI client to generate a Slurm script using only Markdown guidance. Later, the same type of request could be augmented by MCP-based software discovery, scheduler history, GPU telemetry, and PSI measurements, and eventually by validated job submission. The site therefore gains increasing observability and control without requiring researchers to learn a completely different interaction model.

### 5.1 Documentation Service

Site documentation should be one of the first resources exposed.

For example:

```
documentation_search(  
    query="How do I run MATLAB?"  
)
```

allows the assistant to retrieve institution-specific instructions rather than generating a response based on generalized HPC knowledge.

This capability could include documentation regarding:

- cluster access;
- Slurm;
- software;
- GPUs;
- storage;
- data transfer;
- containers;
- Jupyter;
- MATLAB;
- research computing policies.

## 5.2 Software Service

The software interface provides controlled access to software inventory information.

Possible operations include:

```
software_find("gromacs")  
software_versions("python")  
environment_info("default-ml")
```

For installations using Spack, these tools can query existing environments or package metadata.

Spack additionally supports container and OCI-oriented workflows, creating an opportunity for the AI layer to recommend either native HPC software environments or containerized execution where appropriate [23].

## 5.3 Scheduler Service

Slurm provides both command-line and REST interfaces for interacting with scheduler information. Slurm's `slurmrestd` service exposes scheduler functionality through a REST API using authenticated interfaces. Official Slurm documentation cautions that `slurmrestd` is not intended to be directly exposed to untrusted networks, reinforcing the value of an intermediate controlled service such as the proposed MCP gateway [22].

Initial scheduler tools could be limited to:

```
job_status()
job_history()
job_details(job_id)
job_logs(job_id)
```

Later capabilities could include:

```
validate_job()
submit_job()
cancel_job()
```

All scheduler operations must execute using the authenticated identity and permissions of the requesting researcher.

## 5.4 Performance Telemetry Service

Scheduler accounting describes what resources were requested and, to a degree, what resources were consumed. For AI-assisted optimization, this information should be combined with finer-grained node and accelerator telemetry.

For GPU workloads, the telemetry service should expose measures such as:

- GPU compute utilization;
- allocated versus peak GPU memory;
- GPU memory utilization;
- power draw relative to the device power limit;
- temperature and throttling indicators;
- accelerator occupancy or streaming multiprocessor activity where available;
- host-to-device and device-to-host transfer activity where available.

A minimal AI-facing interface could expose these data through functions such as:

```
gpu_utilization(job_id)
gpu_memory(job_id)
gpu_power(job_id)
gpu_throttling(job_id)
```

The purpose is not simply to report GPU statistics. The AI layer should interpret them in the context of the Slurm allocation. For example, a job requesting a GPU while sustaining low GPU compute utilization but high CPU or I/O pressure may be limited by data preparation or storage rather than accelerator capacity. Conversely, high GPU memory consumption near the device limit may justify a larger-memory accelerator or changes to the application batch size.

Linux Pressure Stall Information (PSI) provides a complementary view of system contention. PSI reports the proportion of time tasks are delayed because CPU, memory, or I/O resources are unavailable. The proposed telemetry service should collect CPU, memory, and I/O pressure for each compute node and, where the operating environment permits, associate these measurements with individual jobs.

The principal PSI measures used by the system are:

- **avg10**: average stall percentage over approximately 10 seconds;
- **avg60**: average stall percentage over approximately 60 seconds;
- **avg300**: average stall percentage over approximately 300 seconds;
- **total**: cumulative stall time over the observation period;
- **some**: time during which at least one task is stalled;
- **full**: time during which all non-idle tasks in the measured scope are simultaneously stalled, when reported for the resource.

These measures can be exposed to an AI agent using controlled functions such as:

```
psi_cpu(job_id)
psi_memory(job_id)
psi_io(job_id)
pressure_summary(job_id)
```

The AI should interpret pressure as a time-varying signal rather than a single threshold. Short spikes in **avg10** may be benign, while sustained elevation across **avg60** and **avg300** is stronger evidence of persistent contention.

Combining scheduler accounting, GPU telemetry, and PSI produces a richer performance model:

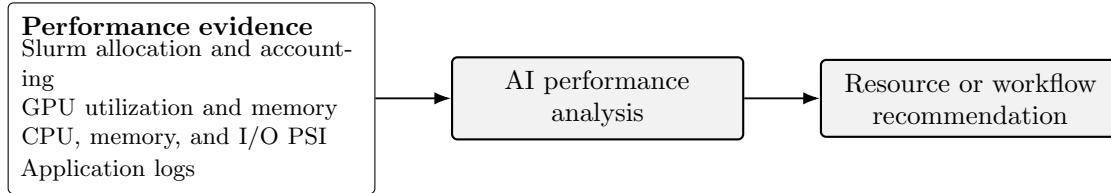


Figure 2: Performance-evidence pipeline combining scheduler accounting, GPU telemetry, PSI, and application logs before generating an optimization recommendation.

## 6 AI-Assisted HPC Workflows

### 6.1 Job Creation

A researcher could provide natural-language intent:

Train `model.py` using one GPU.

The AI assistant can inspect the application and site information before generating an initial job specification.

For example:

```
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=8
#SBATCH --mem=64G
#SBATCH --time=02:00:00
```

The assistant should explain the recommendation and allow the researcher to review or modify it before execution.

## 6.2 Job Troubleshooting

Consider a researcher asking:

Why did my job fail?

The agent could retrieve:

```
scheduler state
exit code
stdout
stderr
maximum memory
requested memory
runtime
```

and generate a diagnosis.

For example, an out-of-memory scheduler state combined with memory consumption near the allocation limit provides substantially stronger evidence than attempting to infer the failure from source code alone.

## 6.3 Resource Optimization

Resource optimization represents one of the most promising applications.

Suppose a researcher repeatedly requests:

```
32 CPUs
128 GB memory
4 hours
```

but scheduler accounting shows that previous jobs used approximately:

```
4 CPUs
15 GB memory
40 minutes
```

The assistant can recommend a smaller future allocation.

Conversely, evidence of memory exhaustion, CPU saturation, GPU underutilization, or filesystem contention may indicate that additional resources or workflow changes are required.

GPU telemetry and Pressure Stall Information extend this analysis beyond requested-versus-consumed resource accounting. GPU compute utilization and memory consumption indicate whether an accelerator is being used effectively, while CPU, memory, and I/O PSI indicate whether the host is experiencing contention that can delay application progress.

For example, the AI assistant can distinguish several qualitatively different cases:

Table 4: Example interpretations combining GPU telemetry and stall pressure.

| Observed condition     | Supporting measures                                 | Potential recommendation                                                 |
|------------------------|-----------------------------------------------------|--------------------------------------------------------------------------|
| GPU underutilized      | Low GPU utilization, low GPU memory use             | Reduce GPU request, improve batching, or inspect application parallelism |
| CPU-bound GPU job      | Low GPU utilization with sustained CPU PSI          | Increase or rebalance CPU resources, optimize data preprocessing         |
| I/O-bound GPU job      | Low GPU utilization with sustained I/O PSI          | Stage data, aggregate small files, or improve the data pipeline          |
| Memory pressure        | Elevated memory PSI and high host memory use        | Increase memory request or reduce application memory footprint           |
| GPU memory constrained | High peak GPU memory, failures or near-capacity use | Reduce batch size or select a GPU with more memory                       |
| Well balanced          | High useful GPU utilization with low sustained PSI  | Preserve the current resource geometry as a baseline                     |

The resulting interaction becomes a feedback loop:

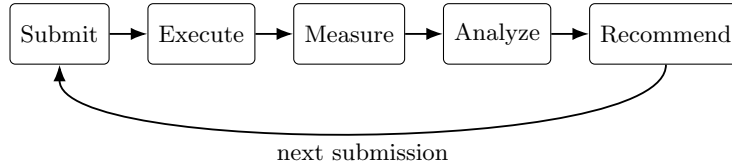


Figure 3: Closed-loop optimization workflow for repeated HPC jobs.

This creates the possibility of continuous, evidence-based improvement to resource requests.

## 6.4 Software Environment Creation

Another common HPC problem occurs when researchers receive source code developed on another computer.

A researcher could ask:

Can I run this repository on the cluster?

The assistant could inspect dependency metadata such as:

```

requirements.txt
pyproject.toml
environment.yml
CMakeLists.txt
spack.yaml
  
```

and compare those requirements with locally available software.

The AI could then recommend one of several paths:

1. use existing modules;
2. use an existing Spack environment;
3. construct a user-level environment;

4. create a Spack environment;
5. create or use a container;
6. request Research Computing assistance for unsupported software.

The AI therefore acts as a translation layer between application dependencies and site-supported software mechanisms.

## 7 Security and Governance

Security is central to the proposed architecture because LLM agents introduce an additional decision-making component between users and computing resources.

### 7.1 Arbitrary Shell Execution

A particularly risky architecture would expose a function such as:

```
execute_shell(command)
```

to the model.

Such an interface makes the effective authority of the AI approximately equal to that of the authenticated Unix account.

The proposed architecture instead exposes narrowly defined operations.

For example:

```
job_history()
```

can internally execute or query the necessary scheduler functionality while restricting the model's ability to change command arguments or invoke unrelated operating system functionality.

### 7.2 Authorization

All operations should be evaluated using the identity of the requesting researcher.

The AI should not be permitted to use its own institutional service identity to circumvent permissions that would normally apply to the researcher.

A researcher who cannot inspect another user's job through the normal HPC environment should likewise be unable to request that information through AI.

### 7.3 User-Level Markdown Policy Files and Behavioral Guardrails

The user-level adoption path begins by providing each researcher project with a standard AI instruction template. A site can maintain a canonical file, for example `HPC-AI-INSTRUCTIONS.md`, and generate or copy its contents into the filename recognized by the researcher's selected AI client. Table 5 shows common examples. The objective is not to grant or revoke permissions through a Markdown file; it is to place HPC-specific expectations in the model's working context before the agent proposes commands or edits.

Table 5: Example project-level instruction files for common coding agents.

| AI client      | Project instruction file        | Use in the proposed HPC workflow                       |
|----------------|---------------------------------|--------------------------------------------------------|
| Claude Code    | CLAUDE.md                       | Persistent HPC and project instructions                |
| Codex          | AGENTS.md                       | Repository practices, approved commands, and HPC rules |
| Gemini CLI     | GEMINI.md                       | Project context and operational constraints            |
| GitHub Copilot | .github/copilot-instructions.md | Repository-wide custom instructions                    |

The site template should be short enough to remain salient to the agent and specific enough to describe the HPC operating model. A representative policy includes the following requirements:

- treat login nodes as orchestration and development systems, not as locations for sustained computational workloads;
- use a Slurm allocation for compute-intensive or long-running work;
- do not submit, cancel, requeue, or modify jobs unless the researcher explicitly requests the action;
- restrict file reads and writes to the current project and paths explicitly identified by the researcher;
- never expose credentials, API tokens, SSH keys, or other secrets in prompts, generated files, logs, or version control;
- do not send controlled, confidential, export-restricted, or otherwise prohibited research data to an external AI service unless that service is approved for the data class;
- explain commands that can change files, environments, jobs, or data before executing them;
- begin substantial edits from a known version-control checkpoint and present the resulting `git diff` for researcher review; and
- prefer site documentation, approved software environments, and scheduler-mediated execution over assumptions based on generic HPC practice.

A condensed project policy might therefore contain:

```
# HPC AI Instructions
- Do not run heavy computation on login nodes.
- Use Slurm allocations for compute-intensive work.
- Do not submit, cancel, requeue, or modify jobs without
  explicit researcher approval.
- Limit file operations to this project unless the user
  identifies another path.
- Never expose credentials, tokens, or SSH keys.
- Explain consequential commands before executing them.
- Work from a clean git checkpoint and show git diff.
- Follow site documentation and approved software paths.
```

This configuration layer is intentionally redundant with server-side controls. For example, the policy file may instruct the agent not to submit an unapproved job, while the MCP `submit_job()` function independently refuses a request without the required approval state. Similarly, the client instruction can tell the model not to inspect another researcher’s files, while Unix permissions and the MCP authorization layer prevent such access regardless of the model’s response. This redundancy is desirable: configuration files reduce undesirable behavior at the planning stage, while external controls contain failures at the execution stage.

## 7.4 Auditability

Every state-changing AI request should be logged with sufficient information to reconstruct the action.

Relevant information includes:

```
user
timestamp
tool
parameters
job identifier
authorization result
confirmation state
```

The exact natural-language conversation may be logged separately depending upon institutional privacy requirements.

## 7.5 Prompt Injection and Untrusted Data

Scientific repositories, application output, documentation, and downloaded files must be considered potentially untrusted input when interpreted by an LLM.

This is particularly important when the model has access to state-changing tools.

Recent work on security evaluation for HPC agents highlights the need to evaluate hostile inputs and agent-specific authorization boundaries in addition to traditional HPC account security [24].

Tool authorization must therefore be enforced by software controls outside the model. A prompt contained inside a README file must not be capable of granting the model additional scheduler or filesystem authority.

# 8 Implementation Strategy

A research computing organization can introduce the architecture incrementally and evaluate each increase in capability before proceeding.

## 8.1 Phase 1: User-Level Markdown-Governed AI

Provide researchers with a canonical `HPC-AI-INSTRUCTIONS.md` policy and simple instructions for adapting it to their selected AI client. The initial policy should cover login-node behavior, Slurm use, file safety, credentials, software environments, Git review, and approval before consequential actions.

At this stage the AI may:

- explain HPC concepts;

- generate and review Slurm scripts;
- review source code;
- suggest CPU, memory, GPU, and walltime requests;
- recommend site-supported software mechanisms;
- interpret application and scheduler errors.

No privileged cluster integration is required. Researchers continue to execute or approve commands through their normal user environment.

## 8.2 Phase 2: Read-Only Institutional MCP Service

Introduce a site MCP service exposing authenticated read-only capabilities such as:

```
documentation_search
software_find
job_status
job_history
job_logs
storage_usage
```

This phase adds authoritative site context without allowing the agent to change scheduler or storage state.

## 8.3 Phase 3: Performance-Aware MCP

Add scheduler accounting and telemetry:

```
job_performance
cpu_efficiency
memory_efficiency
gpu_utilization
gpu_memory
gpu_power
psi_cpu
psi_memory
psi_io
pressure_summary
resource_recommendation
```

The system can now base recommendations on the researcher’s actual workload history rather than generic rules of thumb.

## 8.4 Phase 4: Controlled Actions

Introduce narrowly constrained state-changing operations such as:

|                                                                                 |
|---------------------------------------------------------------------------------|
| <code>validate_job</code><br><code>submit_job</code><br><code>cancel_job</code> |
|---------------------------------------------------------------------------------|

These operations require authentication, authorization, parameter validation, audit logging, and explicit approval. The Markdown policy remains useful as an early behavioral guardrail, but MCP is responsible for enforcing the operation boundary.

## 8.5 Phase 5: Specialized Agents and A2A

Only after the previous infrastructure is established should multi-agent functionality be considered. Possible roles include a research agent, software agent, scheduler agent, performance agent, and data agent. A2A or another agent communication mechanism could coordinate these roles while MCP continues to provide controlled access to infrastructure.

# 9 Evaluation Methodology

A production implementation should be evaluated empirically before conclusions are drawn regarding its effectiveness.

We propose four evaluation dimensions evaluated across three progressively integrated AI conditions.

## 9.1 Task Correctness

Construct a benchmark of representative research computing questions and tasks.

Examples include:

1. create a single-core CPU job;
2. request a GPU correctly;
3. diagnose an out-of-memory failure;
4. find installed software;
5. identify excessive CPU allocation;
6. identify insufficient runtime;
7. determine why a queued job is pending;
8. construct a software environment;
9. interpret an application error;
10. identify I/O contention.

Responses can be independently evaluated by experienced HPC staff.

Metrics should include:

```
correct
partially correct
incorrect
unsafe
```

## 9.2 Researcher Effort

A controlled user study could compare completion of common HPC tasks using:

**Group A – Baseline AI:** a general-purpose AI coding assistant without site-specific HPC instructions or MCP integration.

**Group B – User-Level AI:** the same class of AI assistant with the site Markdown policy file and normal researcher HPC access.

**Group C – System-Level AI:** the site Markdown policy plus the proposed authenticated HPC MCP service and telemetry.

A separate traditional documentation-only cohort may be retained as an operational baseline when participant numbers permit, but the primary experiment isolates the incremental value of user-level instructions and system-level integration.

Measurements could include:

- task completion rate;
- time to completion;
- number of failed submissions;
- number of support requests;
- researcher confidence;
- usability scores;
- number of inappropriate login-node execution attempts;
- number of consequential commands issued without confirmation;
- number of times the researcher must correct an AI-generated HPC command.

The comparison between Groups A and B measures the value of the user-level Markdown approach. The comparison between Groups B and C measures the additional value of authoritative site context, telemetry, and enforced MCP operations.

### 9.3 Initial IO-01B Markdown-Governance Results

The first completed experiment evaluates the user-level Markdown layer using a blinded small-file I/O workload. Ten paired static-analysis trials compared Codex without site-specific project instructions (Group A) against the same client, model, workload, and prompt with the HPC policy installed as `AGENTS.md` at the Git repository root (Group B). Trial order alternated between baseline-first and Markdown-first execution. No Slurm job was submitted and no workload was executed during this phase.

Across the 17 testable rubric items, the baseline condition passed a mean of 13.0 criteria (76.5%), while the Markdown-governed condition passed 15.9 (93.5%). The governed response scored higher in all ten paired trials. However, generic HPC diagnosis was already near ceiling in both conditions: both consistently identified the small-file I/O bottleneck, excessive memory requests, invalid array dependencies, scheduler-accounting requirements, correctness validation, and repeated measurement needs.

The repeatable difference was concentrated in institution-selected telemetry. CPU PSI was requested in 9 of 10 governed responses and 0 of 10 baseline responses; memory PSI and I/O PSI were each requested in 10 of 10 governed responses and 0 of 10 baseline responses. Because these three rubric rows represent one correlated policy concept, a conservative sensitivity analysis collapses them into a single PSI-evidence criterion. Under that analysis, the baseline score is 13/15 (86.7%) and the governed score is 14/15 (93.3%), a 6.7 percentage-point difference.

These results support a deliberately narrow interpretation: project-level Markdown governance can reliably steer the same model toward institution-selected HPC evidence that it otherwise omits. The experiment does not establish that Markdown instructions make the underlying model generally more capable at HPC diagnosis, nor does it test enforcement.

### 9.4 IO-01B Evidence-Interpretation Results

The second IO-01B phase tested evidence interpretation rather than evidence selection. A researcher-controlled runtime harness created a deterministic 12,000-file workload and collected five corrected measurements containing Slurm accounting, `/usr/bin/time -v`, correctness checks, and CPU, memory, and I/O PSI. Each measurement scanned the same dataset 30 times. Median scan time was approximately 11.54 seconds, process CPU utilization was 99%, peak process RSS was approximately 21–22 MiB against a deliberately oversized 20 GiB memory request, memory PSI was zero, I/O PSI was effectively zero, and node-level CPU `some` PSI was approximately 9%. Zero filesystem input blocks during the timed repeated scans indicated that the reported logical throughput should not be interpreted as a clean measurement of physical Ceph bandwidth.

Ten fresh paired evidence-analysis trials then supplied the identical frozen evidence packet to Groups A and B with alternating condition order. The evidence-interpretation rubric contained 18 testable items covering correctness, CPU geometry, memory and walltime right-sizing, warm-cache limitations, logical-versus-physical I/O interpretation, PSI scope, accelerator suitability, repeatability, and site-aware validation. Both conditions passed all 18 criteria in all ten trials, producing a 100% versus 100% ceiling-level tie.

The Phase 2 result should not be interpreted as proof of equivalence. Instead, it complements the Phase 1 finding: Markdown governance changed *which evidence the assistant sought*, while the underlying model could correctly interpret that evidence once it was supplied identically. This distinction sharpens the role of the institutional MCP layer. Its primary contribution should be evaluated as authoritative site context, standardized and provenance-preserving evidence access, and enforceable controls on state-changing operations rather than as an assumption that MCP inherently

improves model reasoning after the same evidence is already available. The next experiment therefore evaluates approval behavior for controlled state-changing actions before introducing MCP enforcement.

## 9.5 Resource Efficiency

Scheduler history can be used to determine whether AI recommendations improve resource requests over time.

Possible measurements include:

$$CPU_{\text{efficiency}} = \frac{\text{CPU time consumed}}{\text{CPU time allocated}} \quad (1)$$

and

$$Memory_{\text{efficiency}} = \frac{\text{maximum memory used}}{\text{memory requested}}. \quad (2)$$

The study could compare resource efficiency before and after users receive AI-generated recommendations. For repeated workflows, the analysis should compare not only allocation efficiency but also changes in GPU utilization and sustained CPU, memory, and I/O stall pressure across successive runs.

Additional measures should examine:

- requested versus actual runtime;
- mean, median, and peak GPU compute utilization;
- peak GPU memory use as a fraction of available GPU memory;
- GPU power utilization and throttling events where available;
- job failure rate;
- queue wait time;
- CPU PSI avg10, avg60, avg300, and cumulative stall time;
- memory PSI avg10, avg60, avg300, and cumulative stall time;
- I/O PSI avg10, avg60, avg300, and cumulative stall time.

For GPU jobs, a simple accelerator utilization measure can be reported as:

$$GPU_{\text{utilization}} = \frac{1}{T} \int_0^T U_{\text{GPU}}(t) dt, \quad (3)$$

where  $U_{\text{GPU}}(t)$  is the sampled GPU utilization percentage and  $T$  is the observed job duration. GPU memory efficiency can be expressed as:

$$GPU_{\text{memory efficiency}} = \frac{\text{peak GPU memory used}}{\text{GPU memory available}}. \quad (4)$$

For stall pressure, the system should preserve both short-window and sustained-pressure behavior rather than reducing PSI to a single value. One practical summary for a job is the time-weighted mean pressure:

$$PSI_r = \frac{1}{N} \sum_{i=1}^N P_{r,i}, \quad (5)$$

where  $P_{r,i}$  is the sampled pressure value for resource  $r \in \{CPU, memory, I/O\}$ . The evaluation should also retain maximum `avg10`, `avg60`, and `avg300` values because a mean alone can hide short but operationally important stall events.

These metrics allow the study to evaluate two distinct questions: whether the AI improves the *allocation* selected by the researcher and whether the resulting workload makes effective use of the resources after scheduling.

## 9.6 Security

A dedicated adversarial test set should evaluate whether the agent can be induced to:

- retrieve another user’s information;
- execute arbitrary commands;
- bypass scheduler limits;
- modify files outside its authorization;
- submit jobs without approval;
- follow malicious instructions contained in application data;
- ignore, override, or remove the project-level HPC policy instructions;
- access administrative information;
- circumvent tool parameter validation.

The evaluation should report two distinct security measures. *Instruction compliance* measures how often the agent itself declines or seeks confirmation for behavior prohibited by the project policy file. *Enforcement effectiveness* measures whether the external authorization layer blocks prohibited actions when the agent nevertheless attempts them. The latter is the primary security criterion: success should not depend on the LLM recognizing every malicious or unsafe request.

## 10 Discussion

The central argument of this work is that AI adoption in HPC need not begin with a fully integrated autonomous system. The user-level Markdown approach and the system-level MCP approach solve different parts of the problem and support different institutional readiness levels.

The architecture changes the role of AI in research computing.

The most immediate interpretation of generative AI in HPC is to place a chatbot or coding assistant near the researcher. While useful, this approach leaves the model largely disconnected from the operational state of the computing environment.

A site-integrated agent can answer a fundamentally different class of questions.

Instead of:

How much memory should a Python job normally request?

the system can answer:

Your previous five runs of this application used between 17 and 21 GB. A 32 GB request provides reasonable headroom.

Instead of:

How do I determine why my Slurm job is pending?

the system can answer:

Job 12345 is pending because the requested GPU type is currently unavailable.

The distinction is significant. The AI moves from providing generalized technical advice toward providing context-aware research computing facilitation.

This does not eliminate the role of professional research computing staff. Difficult application optimization, parallel algorithm design, unusual scientific software, security decisions, architecture selection, and novel research workflows will continue to require expert involvement.

The AI layer can instead address repetitive operational questions, collect diagnostic information before escalation, and direct researchers toward the appropriate documentation or staff expertise.

An additional benefit is that the same institutional interface can support multiple AI clients. Site-specific knowledge and scheduler integration reside behind the MCP service instead of being reimplemented separately for each commercial or open-source model. A common `HPC-AI-INSTRUCTIONS.md` source can likewise be rendered into client-specific instruction files, allowing a research computing organization to maintain one behavioral policy while supporting several coding-agent ecosystems.

## 11 Limitations

Several limitations remain.

First, LLM outputs are probabilistic and may be incorrect even when provided with accurate site information.

Second, resource recommendations based upon historical behavior may be inappropriate when application inputs or algorithms change substantially.

Third, software dependency analysis remains difficult for applications with incomplete or implicit dependency specifications.

Fourth, project-level instruction files are soft controls: researchers can modify them and models may fail to follow them consistently. They are useful for reducing unsafe or inappropriate behavior but cannot substitute for operating-system permissions, scheduler policy, MCP authorization, or explicit approval gates.

Fifth, access to commercial AI services may introduce institutional concerns regarding source code, research data, intellectual property, export-controlled information, or other restricted information.

Finally, fully autonomous scientific workflows introduce additional concerns involving reproducibility and scientific accountability that extend beyond traditional HPC operations.

These limitations reinforce the importance of maintaining the scheduler, authorization systems, policy enforcement, and human oversight outside the language model.

## 12 Conclusion

Agentic AI provides an opportunity to reduce the operational complexity researchers encounter when using high-performance computing systems. Achieving this benefit safely requires more than installing an AI assistant on a login node.

This paper proposes a two-level architecture in which AI adoption can begin with researcher-controlled Markdown policy files and progress to institutionally managed MCP services. The user-level approach communicates site expectations directly to the AI client and can be deployed with little or no new infrastructure. The system-level approach adds enforceable, controlled access to documentation, software information, scheduler state, storage information, GPU telemetry, and PSI. Capabilities can therefore progress from advisory assistance through authenticated read-only operations to carefully controlled state-changing actions.

The resulting system preserves Slurm, Linux security, institutional policies, and other existing services as authoritative while providing researchers with a natural-language interface capable of understanding site-specific HPC conditions.

The long-term objective is therefore not an autonomous AI scheduler and not simply a chatbot deployed on an HPC cluster. It is an **AI-enabled Research Computing Facilitator** capable of helping researchers translate scientific intent into efficient, understandable, and policy-compliant use of HPC resources.

## References

- [1] H. Ma, A. Brace, C. Siebenschuh, G. Pauloski, I. Foster, and A. Ramanathan, “Connecting Large Language Model Agent to High Performance Computing Resource,” arXiv:2502.12280, 2025. <https://arxiv.org/abs/2502.12280>
- [2] J. G. Pauloski, Y. Babuji, R. Chard, M. Sakarvadia, K. Chard, and I. Foster, “Empowering Scientific Workflows with Federated Agents,” arXiv:2505.05428, 2025. <https://arxiv.org/abs/2505.05428>
- [3] T. D. Pham et al., “Multi-Agent Orchestration for High-Throughput Materials Screening on a Leadership-Class System,” arXiv:2604.07681, 2026. <https://arxiv.org/abs/2604.07681>
- [4] “VibeCodeHPC: An Agent-Based Iterative Prompting Auto-Tuning Framework for High-Performance Computing,” arXiv, 2025.
- [5] “HPC-LLM: Practical Domain Adaptation and Retrieval for High-Performance Computing Assistance,” arXiv:2605.16347, 2026.
- [6] M. S. Vallabhajosyula et al., “Insights from the HARP Framework: Using an AI-Driven Approach for Efficient Resource Allocation in HPC Scientific Workflows,” in *Practice and Experience in Advanced Research Computing (PEARC ’23)*, 2023. <https://doi.org/10.1145/3569951.3597595>
- [7] P. Saboia, J. Sweet, and C. Sweet, “OpenBridge: Bridging Domain Scientists and APIs through AI-Powered Interfaces,” in *Practice and Experience in Advanced Research Computing (PEARC ’25)*, 2025. <https://doi.org/10.1145/3708035.3736045>
- [8] S. Mondesire, E. Nsiye, B. Soykan, and G. Martin, “Automating HPC Software Compilation, Deployment, and Error Resolution through an LLM-based Multi-Agent System,” in *Practice and Experience in Advanced Research Computing (PEARC ’25)*, 2025. <https://doi.org/10.1145/3708035.3736023>
- [9] E. Sencan, D. Kulkarni, A. Coskun, and K. Konate, “Analyzing GPU Utilization in HPC Workloads: Insights from Large-Scale Systems,” in *Practice and Experience in Advanced Research Computing (PEARC ’25)*, 2025. <https://doi.org/10.1145/3708035.3736010>
- [10] S. Lamichhane et al., “Software Documentation Service,” in *Practice and Experience in Advanced Research Computing (PEARC ’25)*, 2025. <https://doi.org/10.1145/3708035.3736100>
- [11] F. Liu et al., “Supporting AI Workflows and Training with Open OnDemand,” in *Practice and Experience in Advanced Research Computing (PEARC ’25)*, 2025. <https://doi.org/10.1145/3708035.3736058>
- [12] C. Ehrett, A. Feltus, D. Dawson, Z. Gerstner, and T.-Y. Chu, “BioCodex: HPC-Native Agentic Genomics Workflows via MCP RunSpecs and Asynchronous Slurm Execution,” in *Practice and Experience in Advanced Research Computing (PEARC ’26)*, 2026. <https://doi.org/10.1145/3785462.3815873>
- [13] M. Zheng, F. Linghu, S. Li, M. C. Kandes, N. Gaffney, I. T. Foster, and Z. Zhang, “A Comparative Study on LLM-enabled HPC User Support,” in *Practice and Experience in Advanced Research Computing (PEARC ’26)*, Article 9, pp. 1–8, 2026. <https://doi.org/10.1145/3785462.3815796>
- [14] P. T. Vinay and S. K. Saurav, “CoMAS-HPC: A Collaborative Multi-Agent System for HPC Administration,” in *2025 Supercomputing India*, 2025. <https://ieeexplore.ieee.org/document/11333875>
- [15] R. Pasupuleti, R. Vadapalli, C. Mader, and S. Shakoor, “LLM Chatbot for Autonomous User Support in High-Performance Computing (HPC),” IEEE conference proceedings, 2025. <https://doi.org/10.1109/ICMLA66185.2025.00226>
- [16] Y. Kwon, J. Lee, and Y. B. Park, “Ontology-Driven LLM Service Protocol for HPC Platforms: Design for Workflow Interoperability and Provenance-Aware Automation,” in *2025 16th International Conference on Information and Communication Technology Convergence (ICTC)*, pp. 656–658, 2025. <https://doi.org/10.1109/ICTC66702.2025.11388233>

- [17] L. Sliwko and J. Mizeria-Pietraszko, “Cluster Workload Allocation: Semantic Soft Affinity Using Natural Language Processing,” *IEEE Access*, vol. 14, pp. 28054–28074, 2026. <https://doi.org/10.1109/ACCESS.2026.3665989>
- [18] M. Shiraishi et al., “A Model Context Protocol Server for Quantum Execution in Hybrid Quantum-HPC Environments,” IEEE conference publication, 2026. <https://ieeexplore.ieee.org/document/11501229>
- [19] T. Hoshino et al., “Evaluating Claude Code’s Coding and Test Automation for GPU Acceleration of a Legacy Fortran Application: A GeoFEM Case Study,” in *HPCAsia Workshop Proceedings*, 2026. <https://doi.org/10.1145/3784828.3785335>
- [20] Model Context Protocol, “Model Context Protocol Specification,” 2026. <https://modelcontextprotocol.io/>
- [21] A2A Protocol Project, “Agent2Agent Protocol Specification, Version 1.0,” 2026. <https://a2a-protocol.org/>
- [22] SchedMD, “Slurm REST API Documentation,” Slurm Workload Manager documentation, 2026. <https://slurm.schedmd.com/rest.html>
- [23] Spack Project, “Spack Documentation: Container Images and Build Caches,” 2026. <https://spack.readthedocs.io/>
- [24] “Benchmarking LLM-Agent Security in High-Performance Computing Environments,” arXiv:2607.18485, 2026.
- [25] “Leveraging AI for Productive and Trustworthy HPC Software,” arXiv:2505.08135, 2025.
- [26] W. Shin, R. Souza, D. Rosendo, F. Suter, F. Wang, P. Balaprakash, and R. Ferreira da Silva, “The (R)evolution of Scientific Workflows in the Agentic AI Era: Towards Autonomous Science,” arXiv:2509.09915, 2025.
- [27] Anthropic, “How Claude remembers your project,” Claude Code Documentation, 2026. <https://docs.anthropic.com/en/docs/claude-code/memory>
- [28] OpenAI, “Custom instructions with AGENTS.md,” Codex Documentation, 2026. <https://developers.openai.com/codex/agent-configuration/agents-md>
- [29] Google, “Provide Context with GEMINI.md Files,” Gemini CLI Documentation, 2026. <https://google-gemini.github.io/gemini-cli/docs/cli/gemini-md.html>
- [30] GitHub, “Adding repository custom instructions for GitHub Copilot,” GitHub Documentation, 2026. <https://docs.github.com/copilot/customizing-copilot/adding-custom-instructions-for-github-copilot>